

Documentazione Progetto Sistemi Operativi: PandOSsh Fase 1

Luca Argentino, Milo Disalvatore, Matteo Mazzetti, Zeno Maccari

17 maggio 2026

1 Gestione PCB (pcb.c)

Il modulo `pcb.c` si occupa della gestione dei **Process Control Blocks (PCB)**, occupandosi della loro allocazione/deallocazione e del mantenimento delle strutture a coda e ad albero.

1.1 Strutture Dati Globali

Il modulo utilizza le seguenti variabili statiche globali per gestire il pool di PCB:

- `static pcb_t pcbFree_table[MAXPROC]`: Array statico che contiene i PCB disponibili per il sistema.
- `static struct list_head pcbFree_h`: Testa della lista che collega i PCB attualmente non in uso (liberi).
- `static int next_pid`: Contatore intero utilizzato per assegnare un Identificatore di Processo (PID) unico a ogni nuovo PCB allocato.

1.2 Descrizione delle Funzioni di Allocazione

1.2.1 `void initPcbs()`

1. Inizializza la testa della lista dei PCB liberi (`pcbFree_h`) tramite la macro `INIT_LIST_HEAD`.
2. Itera attraverso l'array statico `pcbFree_table` e inserisce ogni elemento nella lista `pcbFree_h` utilizzando il campo `p_list`.

1.2.2 `void freePcb(pcb_t *p)`

1. Inserisce il PCB puntato da `p` nella lista dei PCB liberi (`pcbFree_h`), rendendolo disponibile per future allocazioni.

1.2.3 `pcb_t *allocPcb()`

1. Controlla se la lista `pcbFree_h` è vuota. Se sì, ritorna `NULL`.
2. Se è presente un PCB libero:
 - Rimuove il primo elemento dalla lista `pcbFree_h`.
 - Inizializza tutti i campi del PCB a `NULL` o 0 (inclusi i puntatori di lista e di stato).
 - Assegna al campo `p_pid` il valore corrente di `next_pid` e incrementa quest'ultimo.
 - Ritorna il puntatore al PCB inizializzato.

1.3 Descrizione delle Funzioni per Code di Processi

1.3.1 `int emptyProcQ(struct list_head *head)`

1. Ritorna `TRUE` se la lista puntata da `head` è vuota, `FALSE` altrimenti.

1.3.2 `void insertProcQ(struct list_head *head, pcb_t *p)`

1. Memorizza la priorità del PCB puntato da `p`.
2. Scorre tutta la lista associata ad `head` e confronta le priorità dei PCB che incontra con quella memorizzata.
3. Appena incontra un PCB con priorità minore di quella di `p` inserisce il PCB `p`.

1.3.3 `pcb_t *headProcQ(struct list_head *head)`

1. Controlla se l'elemento sentinella `head` punta ad una coda nulla e in tal caso ritorna `NULL`.
2. Se così non fosse allora restituisce il primo PCB della lista.

1.3.4 `pcb_t *removeProcQ(struct list_head *head)`

1. Se `head` punta ad una coda vuota allora viene ritornato `NULL`.
2. Altrimenti salva temporaneamente il primo PCB per ritornarlo.
3. Rimuove il primo PCB.
4. Restituisce il PCB appena salvato, ossia quello rimosso.

1.3.5 `pcb_t *outProcQ(struct list_head *head, pcb_t *p)`

1. Scorre tutta la lista puntata da `head` alla ricerca del PCB di cui `p` è il puntatore.
 - Se trova `p` allora lo rimuove e lo restituisce.
 - Se `p` non è presente nella lista allora ritorna `NULL`.

1.4 Descrizione delle Funzioni per Alberi di Processi

1.4.1 `int emptyChild(pcb_t *p)`

1. Verifica se il PCB puntato da `p` ha dei figli controllando se la lista `p_child` è vuota.
2. Ritorna `TRUE` se la lista è vuota (o se `p` è `NULL`), altrimenti ritorna `FALSE`.

1.4.2 `void insertChild(pcb_t *prnt, pcb_t *p)`

1. Se entrambi i puntatori sono validi, imposta `prnt` come padre del PCB `p` (campo `p_parent`).
2. Inserisce il PCB `p` nella lista dei figli del padre (`prnt->p_child`) utilizzando il campo `p_sib`.

1.4.3 `pcb_t *removeChild(pcb_t *p)`

1. Controlla se il PCB `p` ha figli tramite `emptyChild`. Se non ne ha, ritorna `NULL`.
2. Altrimenti, individua il primo figlio nella lista `p_child`.
3. Rimuove tale figlio dalla lista e ne imposta il puntatore al padre (`p_parent`) a `NULL`.
4. Ritorna il puntatore al PCB del figlio rimosso.

1.4.4 `pcb_t *outChild(pcb_t *p)`

1. Verifica se il PCB `p` ha effettivamente un padre. Se non lo ha, ritorna `NULL`.
2. Rimuove il PCB `p` dalla lista dei fratelli (e quindi dalla lista dei figli del padre) tramite il campo `p_sib`.
3. Resetta il puntatore `p_parent` a `NULL`.
4. Ritorna il puntatore `p`.

2 Gestione Semafori (asl.c)

Il modulo `asl.c` implementa la gestione della **Active Semaphore List (ASL)**.

La struttura dati ASL è utilizzata per la gestione della lista dei semafori attivi, ovvero quei semafori in cui almeno un PCB è bloccato.

2.1 Strutture Dati Globali

Il modulo utilizza le seguenti variabili statiche globali per gestire lo stato dei semafori:

- `static semd_t semd_table[MAXPROC]`: Array statico contenente i semafori liberi con dimensione massima (`MAXPROC`).
- `static struct list_head semdFree_h`: Testa della lista che collega i descrittori di semafori liberi.
- `static struct list_head semd_h`: Testa della lista che rappresenta la ASL che contiene i descrittori dei semafori attivi.

2.2 Descrizione delle Funzioni

2.2.1 `void initASL()`

1. Inizializza le teste delle liste `semdFree_h` e `semd_h` utilizzando la macro `INIT_LIST_HEAD`.
2. Itera attraverso l'array `semd_table` e inserisce ogni elemento nella lista dei descrittori liberi (`semdFree_h`) utilizzando il campo `s_link`.

2.2.2 `int insertBlocked(int* semAdd, pcb_t* p)`

1. Scorre la ASL (`semd_h`) per cercare un semaforo con chiave uguale a `semAdd`.
2. **Se il semaforo esiste:**

- Inserisce il PCB `p` in coda alla lista processi del semaforo (`s_procq`).
- Aggiorna il puntatore `p->p_semAdd` del PCB.
- Ritorna `FALSE`.

3. **Se il semaforo non esiste:**

Controlla se la lista dei liberi `semdFree_h` è vuota. Se sì, ritorna `TRUE`.

Altrimenti:

- Rimuove il primo semaforo disponibile dalla `semdFree_h`.
- Inizializza il nuovo semaforo: imposta la chiave `s_key`, inizializza la coda processi `s_procq`.

- Aggiunge il nuovo semaforo in coda alla ASL (`semAdd_h`).
- Aggiunge il PCB `p` alla coda del nuovo semaforo e aggiorna il riferimento `p->p_semAdd`.
- Ritorna `FALSE`.

2.2.3 `pcb_t* removeBlocked(int* semAdd)`

1. Cerca il semaforo con chiave `semAdd` nella ASL.
2. Se non trovato, ritorna `NULL`.
3. Se trovato:
 - Identifica e rimuove il primo PCB nella coda `s_procq`.
 - Controlla se la coda processi del semaforo è diventata vuota (`emptyProcQ`).
 - Se la coda è vuota, il semaforo viene disattivato: viene rimosso dalla ASL e reinserito nella lista dei liberi `semFree_h`.
 - Ritorna il puntatore al PCB rimosso.

2.2.4 `pcb_t* outBlocked(pcb_t* p)`

1. Scorre la ASL per trovare il semaforo corrispondente a `p->p_semAdd`.
2. Una volta trovato il semaforo, scorre la sua coda processi (`s_procq`) per trovare il PCB `p`.
3. Se il PCB viene trovato:
 - Rimuove il PCB dalla lista.
 - Verifica se la coda del semaforo è diventata vuota. Se sì, rimuove il descrittore del semaforo dalla ASL e lo restituisce alla lista dei liberi.
 - Ritorna il puntatore a `p`.
4. Se il semaforo non viene trovato o il PCB non è nella coda, ritorna `NULL`.

2.2.5 `pcb_t* headBlocked(int* semAdd)`

1. Cerca il semaforo con chiave `semAdd` nella ASL.
2. Se trovato, utilizza la funzione `headProcQ` (definita in `pcb.c`) per restituire il primo elemento della coda processi.
3. Se il semaforo non esiste, ritorna `NULL`.